Nama : Fadiyah Nurilwalid Hajar

NRP : 3125522012

Mata Kuliah : Pemrograman Berorientasi Objek (PBO)

Program Studi : PSDKU PENS di Sumenep

Laporan Praktikum Modul 6: Polymorphism, Method Overriding, Method Overloading, dan Dynamic Binding

Halaman 1: Perencanaan & Sprint Backlog

1. Nama Project

Sistem Penjualan dan Manajemen Data Toko

2. Sprint Goal

Mengembangkan behavior objek pada proyek melalui method overriding dan polymorphism sehingga subclass dapat merespons pemanggilan method yang sama dengan perilaku yang berbeda.

3. Sprint Backlog

ID	*Sprint Backlog*	*Status*
SB-01	*Audit superclass dan subclass P5 (Person dan Pelanggan)*	*DONE*
SB-02	*Menentukan behavior yang dapat dioverride (tampilkanPeran())*	*DONE*
SB-03	*Implementasi overriding pada subclass (Pelanggan dan PegawaiToko)*	*DONE*
SB-04	*Implementasi overloading sederhana (cariData pada class Main)*	*DONE*
SB-05	*Implementasi upcasting (Person p = new Pelanggan(...))*	*DONE*
SB-06	*Membuat polymorphic collection (array bertipe Person)*	*DONE*
SB-07	*Melakukan pengujian dynamic binding*	*DONE*

4. Hierarchy Class P5 & P6

- *Superclass: Person (memiliki atribut umum: nama, alamat)*

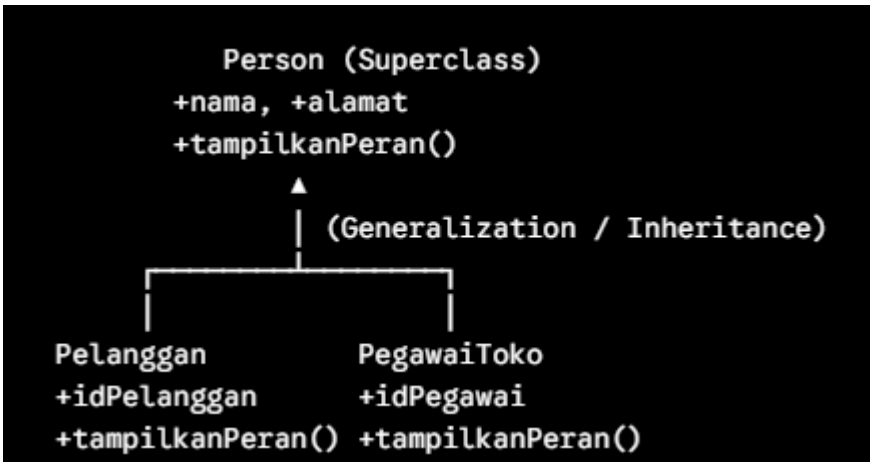
- *Subclass 1: Pelanggan (turunan Person dengan tambahan atribut idPelanggan)*
- *Subclass 2: PegawaiToko (turunan Person dengan tambahan atribut idPegawai)*

5. Behavior yang Dipilih untuk Polymorphism

Method tampilkanPeran() yang didefinisikan secara umum di dalam superclass Person dan di-override secara spesifik pada masing-masing subclass (Pelanggan dan PegawaiToko) untuk mencetak informasi peran yang berbeda.

Halaman 2: Implementasi & Desain Program

1. *Class Diagram Terbaru*



2. *Contoh Potongan Kode*
   - *Method Overriding (Pelanggan.java):*

```java
@Override
public void tampilkanPeran() {
    System.out.println("Peran: Pelanggan Toko (ID: " + this.idPelanggan + ")");
}
```

   - *Method Overloading (Main.java):*

```java
public static void cariData(String nama) {
    System.out.println("[Pencarian] Mencari data berdasarkan Nama: " + nama);
}

public static void cariData(int idInt) {
    System.out.println("[Pencarian] Mencari data berdasarkan ID (Angka): " + idInt);
}
```

   - *Upcasting & Polymorphic Reference (Main.java):*

```java
Person p1 = new Pelanggan("P001", "Fadiyah Nurilwalid", "Surabaya");
Person p2 = new PegawaiToko("EMP01", "Budi Santoso", "Sidoarjo");
```

Halaman 3: Pengujian, Sprint Review, & Retrospective

```
========================================
   PENGUJIAN POLYMORPHISM & DYNAMIC BINDING
========================================

--- [BAGIAN C: METHOD OVERLOADING] ---
[Pencarian] Mencari data berdasarkan Nama: Fadiyah Nurilwalid
[Pencarian] Mencari data berdasarkan ID (Angka): 101

--- [BAGIAN D: UPCASTING & POLYMORPHIC REFERENCE] ---
Peran: Pelanggan Toko (ID: P001)
Peran: Pegawai / Staf Toko (ID: EMP01)

--- [BAGIAN E & F: POLYMORPHIC COLLECTION & DYNAMIC BINDING] ---
Peran: Pelanggan Toko (ID: P002)
Peran: Pegawai / Staf Toko (ID: EMP02)
Peran: Pelanggan Toko (ID: P003)
```

1. *Tabel Pengujian Dynamic Binding*

Reference	*Object Aktual*	*Method yang Dipanggil*	*Output yang Dihasilkan*
Person	*Pelanggan*	*tampilkanPeran()*	*Peran: Pelanggan Toko (ID: P002)*
Person	*PegawaiToko*	*tampilkanPeran()*	*Peran: Pegawai / Staf Toko (ID: EMP02)*

Penjelasan Dynamic Binding:

Meskipun variabel referensi (p) bertipe Person, method yang dieksekusi saat program berjalan (runtime) adalah method milik objek aslinya (Pelanggan atau PegawaiToko). Hal ini terjadi karena Java menerapkan Dynamic Method Dispatch.

2. *Sprint Review*

Item Penilaian	*Hasil / Status*
Superclass dan subclass tersedia	*Berhasil*
Method overriding berhasil	*Berhasil*
Minimal 2 subclass memiliki behavior berbeda	*Berhasil*
Method overloading tersedia	*Berhasil*
Upcasting berhasil	*Berhasil*
Polymorphic collection berhasil	*Berhasil*
Dynamic binding dapat dibuktikan	*Berhasil*
Program berhasil dikompilasi & dijalankan	*Berhasil*

3. Sprint Retrospective

- What Went Well?

Penerapan konsep polymorphic collection menggunakan array objek Person berhasil diimplementasikan dengan mulus, dan pembuktian dynamic binding berjalan sesuai ekspektasi praktikum.

- What Went Wrong?

Sempat ada sedikit kebingungan dalam menentukan parameter yang tepat saat membuat method overloading, namun berhasil diselesaikan dengan membedakan tipe data parameter (String vs int).

- Improvement:

Struktur hierarki kelas sudah cukup baik dan siap untuk dikembangkan lebih lanjut ke materi berikutnya, yaitu penerapan Abstract Class dan Interface pada Modul 7.